hw2

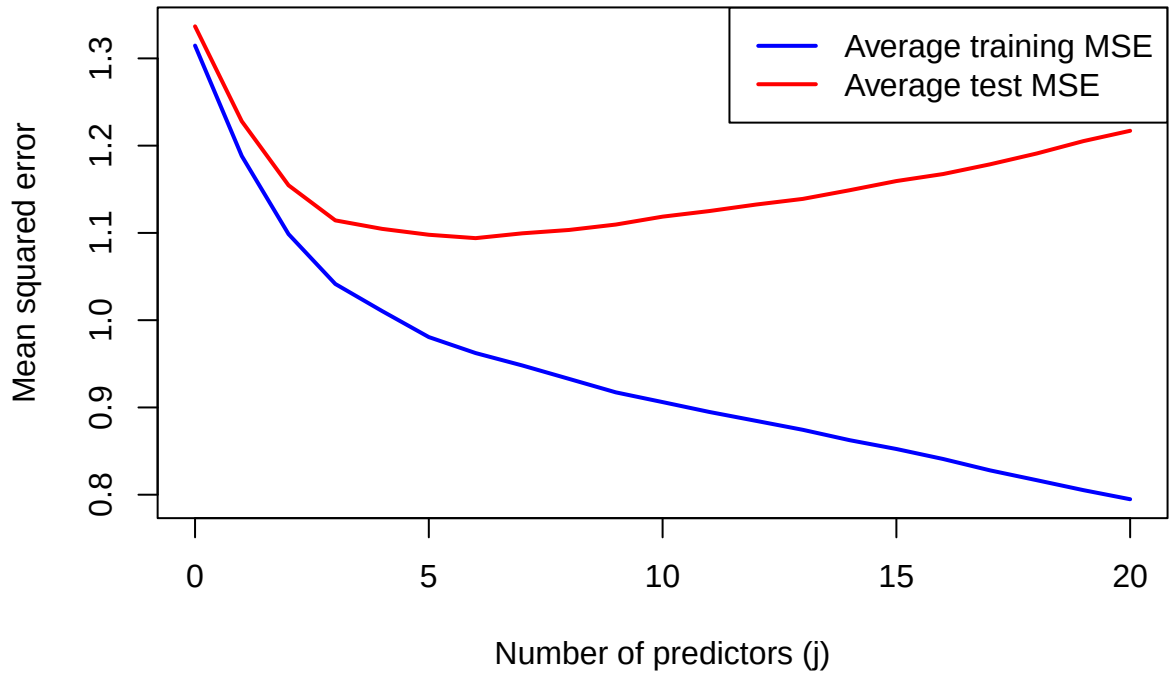
zhijing huang

2026-09-05

Question 1

a. the average training and test MSE over the 200 simulation runs are displayed as the following table, as well as the plots:

```
set.seed(43202)
n <- 100
q <- 20
repetitions <- 200
X_all <- matrix(
  rnorm(n * q, mean = 0, sd = 1),
  nrow = n,
  ncol = q
)
k <- 1:q
beta <- 0.4 ^ sqrt(k)
mu <- drop(X_all %*% beta)
train_mse <- test_mse <- matrix(NA_real_, repetitions, q + 1)

q <- ncol(X_all)
theory_train <- theory_test <- numeric(q + 1)

for (simulation in seq_len(repetitions)) {
  y_train <- mu + rnorm(n)
  y_test  <- mu + rnorm(n)

  for (j in 0:q) {
    if (j == 0) {
      X_j <- matrix(1, nrow = n, ncol = 1)
    } else {
      X_j <- cbind(1, X_all[, 1:j, drop = FALSE])
    }

    model <- lm.fit(x = X_j, y = y_train)
    fitted_values <- model$fitted.values

    train_mse[simulation, j + 1] <-
      mean((y_train - fitted_values)^2)

    test_mse[simulation, j + 1] <-
      mean((y_test - fitted_values)^2)
  }
}

average_train_mse <- colMeans(train_mse)
average_test_mse  <- colMeans(test_mse)

plot(
  0:q,
  average_train_mse,
  type = "l",
  col = "blue",
  lwd = 2,
  ylim = range(average_train_mse, average_test_mse),
  xlab = "Number of predictors (j)",
  ylab = "Mean squared error"
)

lines(
  0:q,
  average_test_mse,
  col = "red",
  lwd = 2
)

legend(
  "topright",
  legend = c("Average training MSE", "Average test MSE"),
  col = c("blue", "red"),
  lwd = 2
)
```

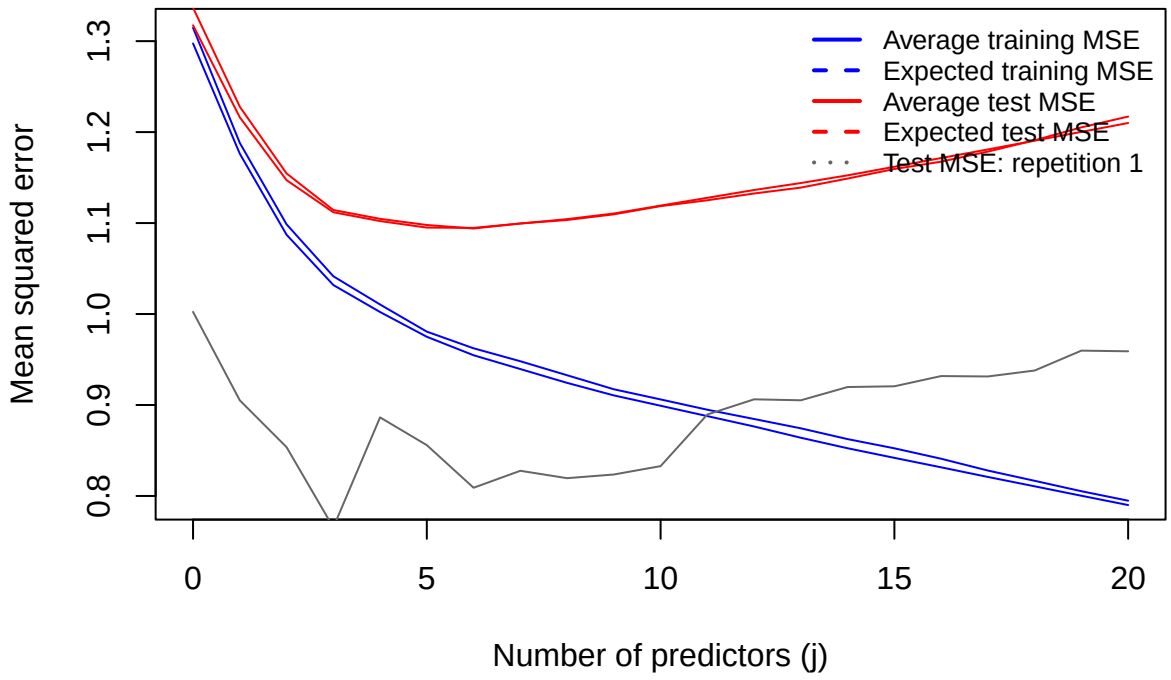



```
data.frame(
  j = 0:q,
  train = colMeans(train_mse),
  test = colMeans(test_mse)
)
```

```
##      j     train     test
## 1    0 1.3146643 1.336716
## 2    1 1.1880807 1.227913
## 3    2 1.0986162 1.154570
## 4    3 1.0415172 1.114369
## 5    4 1.0104722 1.104631
## 6    5 0.9806254 1.097872
## 7    6 0.9623332 1.094007
## 8    7 0.9480987 1.099547
## 9    8 0.9327112 1.103380
## 10   9 0.9172755 1.109580
## 11  10 0.9061281 1.118629
## 12  11 0.8948077 1.125064
## 13  12 0.8846025 1.132503
## 14  13 0.8743504 1.138992
## 15  14 0.8624938 1.148920
## 16  15 0.8523713 1.159433
## 17  16 0.8409206 1.167473
## 18  17 0.8279270 1.178563
## 19  18 0.8166393 1.190993
## 20  19 0.8052183 1.205204
## 21  20 0.7948152 1.217043
```

b,c The simulation averages should be similar to their expectations, and the training error will continue to decrease, as each new model inherits the old one. Expected test MSE contains the mean squared approximation bias B_j^2/n, as n goes bigger, the error grows smaller, and the estimation-variance term p_j/n, which increases as n increases. Because the test MSE curve retains the random variaation from the response vector, it is less smooth than the average test MSE. Because the testing curve follows training noise, so it cannot be used by itself to select the predictor count.

```r
for (j in 0:q) {
  if (j == 0) {
    X_j <- matrix(1, nrow = n, ncol = 1)
  } else {
    X_j <- cbind(1, X_all[, 1:j, drop = FALSE])
  }

  fit_mu <- lm.fit(x = X_j, y = mu)
  projected_mu <- fit_mu$fitted.values

  bias_mse <- mean((mu - projected_mu)^2)
  p_j <- ncol(X_j)

  theory_train[j + 1] <- bias_mse + 1 - p_j / n
  theory_test[j + 1]  <- bias_mse + 1 + p_j / n
}
first_test_mse <- test_mse[1, ]
plot( 0:q, average_train_mse, type = "l", col = "blue",
  xlab = "Number of predictors (j)",
  ylab = "Mean squared error"
)

lines( 0:q, theory_train,col = "blue")

lines(0:q, average_test_mse, col = "red")

lines(0:q, theory_test, col = "red")

lines(0:q, first_test_mse, col = "gray40")

legend(
  "topright",
  legend = c(
    "Average training MSE",
    "Expected training MSE",
    "Average test MSE",
    "Expected test MSE",
    "Test MSE: repetition 1"
  ),
  col = c("blue", "blue", "red", "red", "gray40"),
  lty = c(1, 2, 1, 2, 3),
  lwd = 2,
  bty = "n",
  cex = 0.8
)
```



Question 2

a. the target mean is 1.

```
x0 <- c(0, 1, 0, 0, 1, 0)
beta <- rep(0.5, q)

mu0 <- sum(x0 * beta)
```

```
## Warning in x0 * beta: longer object length is not a multiple of shorter object
## length
```

```
mu0
```

```
## [1] 3.5
```

b. The smallest theoretical error is when j=5.

```
set.seed(43203)
n <- 100
q <- 6
sigma2 <- 1
repetitions <- 200
beta <- rep(0.5, q)
x0 <- c(0, 1, 0, 0, 1, 0)
mu0 <- sum(x0 * beta)

X_all <- outer(
  1:n,
  1:q,
  function(i, k) {
    sqrt(2) * cos(pi * k * (i - 0.5) / n)
  }
)

simulation_error <- matrix(
  NA_real_,
  nrow = repetitions,
  ncol = q + 1
)

# True mean
mu <- drop(X_all %*% beta)

for (simulation in seq_len(repetitions)) {
  y <- mu + rnorm(n)
  for (j in 0:q) {
    X_j <- cbind(1, X_all[, seq_len(j), drop = FALSE])
    coefficients <- solve(crossprod(X_j), crossprod(X_j, y))
    target_row <- c(1, x0[seq_len(j)])
    prediction <- drop(target_row %*% coefficients)
    simulation_error[simulation, j + 1] <-(prediction - mu0)^2
  }
}

simulated <- colMeans(simulation_error)
simulated
```

```
## [1] 1.01891864 1.01891864 0.26711699 0.26711699 0.26711699 0.03375873 0.03375873
```

c. Although all six regression coefficients are nonzero, the mean response at the target point depends only on predictors 2 and 5. This is because the error changes only when predictor 2 or 5 is added, and the other targeted fitted coordinates are 0. This error is target-specific because it measures prediction performance only at x sub0. Consequently, including predictors whose coordinates are zero at this target does not change the theoretical prediction error for this design.

Question 3

a.

For the training error,

$$\mathbf{y} - \mathbf{H}\mathbf{y} = (\mathbf{I}_n - \mathbf{H})\mathbf{y}.$$

Substituting $\mathbf{y} = \boldsymbol{\mu} + \boldsymbol{\epsilon}$ gives

$$\mathbf{y} - \mathbf{H}\mathbf{y} = (\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu} + (\mathbf{I}_n - \mathbf{H})\boldsymbol{\epsilon}.$$

Using $\mathbf{r} = (\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu}$, this becomes

$$\mathbf{y} - \mathbf{H}\mathbf{y} = \mathbf{r} + (\mathbf{I}_n - \mathbf{H})\boldsymbol{\epsilon}.$$

Therefore,

$$\begin{aligned}\|\mathbf{y} - \mathbf{H}\mathbf{y}\|^2 = \|\mathbf{r}\|^2 + 2\mathbf{r}^\mathsf{T}(\mathbf{I}_n - \mathbf{H})\boldsymbol{\epsilon} \\ + \boldsymbol{\epsilon}^\mathsf{T}(\mathbf{I}_n - \mathbf{H})^\mathsf{T}(\mathbf{I}_n - \mathbf{H})\boldsymbol{\epsilon}.\end{aligned}$$

$$\|\mathbf{r}\|^2 = B^2$$

$$(\mathbf{I}_n - \mathbf{H})^\mathsf{T}(\mathbf{I}_n - \mathbf{H}) = \mathbf{I}_n - \mathbf{H}.$$

The expected noise term is

$$\sigma^2 \operatorname{tr}(\mathbf{I}_n - \mathbf{H}),$$

$$\operatorname{tr}(\mathbf{I}_n - \mathbf{H}) = n - p.$$

Thus, the expected residual sum of squares is

$$B^2 + \sigma^2(n - p).$$

Dividing by n gives

$$\boxed{E(\mathrm{MSE}_{\text{train}} \mid \mathbf{X}) = \frac{B^2}{n} + \sigma^2\left(1 - \frac{p}{n}\right)}.$$

For the test error,

$$\begin{aligned}\mathbf{y}^* - \mathbf{H}\mathbf{y} &= \boldsymbol{\mu} + \boldsymbol{\epsilon}^* - \mathbf{H}(\boldsymbol{\mu} + \boldsymbol{\epsilon}) \\ &= (\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu} + \boldsymbol{\epsilon}^* - \mathbf{H}\boldsymbol{\epsilon} \\ &= \mathbf{r} + \boldsymbol{\epsilon}^* - \mathbf{H}\boldsymbol{\epsilon}.\end{aligned}$$

The three components are the approximation error $\mathbf{r}$, the new test noise $\boldsymbol{\epsilon}^*$, and the training estimation noise $-\mathbf{H}\boldsymbol{\epsilon}$. When taking expectations, all cross-products have expectation zero. In particular, $\boldsymbol{\epsilon}^*$ and $\boldsymbol{\epsilon}$ are independent. Thus,

$$\begin{aligned}E\big(\|\mathbf{y}^* - \mathbf{H}\mathbf{y}\|^2 \mid \mathbf{X}\big) = B^2 + E(\|\boldsymbol{\epsilon}^*\|^2 \mid \mathbf{X}) \\ + E(\boldsymbol{\epsilon}^\mathsf{T}\mathbf{H}^\mathsf{T}\mathbf{H}\boldsymbol{\epsilon} \mid \mathbf{X}).\end{aligned}$$

The test-noise term is

$$E(\|\boldsymbol{\epsilon}^*\|^2 \mid \mathbf{X}) = n\sigma^2.$$

Because $\mathbf{H}^\mathsf{T}\mathbf{H} = \mathbf{H}^2 = \mathbf{H}$, the estimation-noise term is

$$\sigma^2 \operatorname{tr}(\mathbf{H}) = p\sigma^2.$$

$$E\big(\|\mathbf{y}^* - \mathbf{H}\mathbf{y}\|^2 \mid \mathbf{X}\big) = B^2 + n\sigma^2 + p\sigma^2.$$

$$\boxed{E(\mathrm{MSE}_{\text{test}} \mid \mathbf{X}) = \frac{B^2}{n} + \sigma^2\left(1 + \frac{p}{n}\right)}.$$

b.

$$\begin{aligned}E(\mathrm{MSE}_{\text{test}} - \mathrm{MSE}_{\text{train}} \mid \mathbf{X}) &= \sigma^2\left(1 + \frac{p}{n}\right) - \sigma^2\left(1 - \frac{p}{n}\right) \\ &= \boxed{\frac{2p\sigma^2}{n}}.\end{aligned}$$

For $n = 120$, $p = 5$, $B^2/n = 0.04$, and $\sigma^2 = 1$,

$$E(\text{MSE}_{\text{train}} \mid \mathbf{X}) = 0.04 + 1\left(1 - \frac{5}{120}\right)$$
$$= \boxed{0.9983},$$

$$E(\text{MSE}_{\text{test}} \mid \mathbf{X}) = 0.04 + 1\left(1 + \frac{5}{120}\right)$$
$$= \boxed{1.0817}.$$

The expected difference is

$$1.0817 - 0.9983 = 0.0833.$$

c.

$$\widehat{\text{MSE}}_{\text{test}} = \frac{\text{RSS} + 2p\widehat{\sigma}^2}{n}$$
$$= \frac{116.4 + 2(5)(0.96)}{120}$$
$$= \frac{126}{120}$$
$$= \boxed{1.05}.$$

$$C_p = \frac{\text{RSS}}{\widehat{\sigma}^2} - n + 2p$$
$$= \frac{116.4}{0.96} - 120 + 10$$
$$= 121.25 - 120 + 10$$
$$= \boxed{11.25}.$$

$$C_p = \frac{\text{RSS}}{\widehat{\sigma}^2} - n + 2p,$$

$$C_p + n = \frac{\text{RSS}}{\widehat{\sigma}^2} + 2p.$$

$$\frac{\widehat{\sigma}^2(C_p + n)}{n} = \frac{0.96(11.25 + 120)}{120}$$
$$= 1.05$$
$$= \frac{\text{RSS} + 2p\widehat{\sigma}^2}{n}.$$

Question 4

a.

```
diabetes <- read.csv("../week-02/data/diabetes.csv")
training <- diabetes[1:370, ]
n <- nrow(training)
model_A <- lm(
  y ~ bmi + bp + s5 + sex + s1 + s2 + s4,
  data = training
)
p_A <- length(coef(model_A))
p_A
```

```
## [1] 8
```

```
rss_A <- sum(residuals(model_A)^2)

model_B <- lm(
  y ~ bmi + bp + s5 + sex + s1 + s2 + s4 + s6,
  data = training
)
p_B <- length(coef(model_B))
rss_B <- sum(residuals(model_B)^2)
```

```r
model_full <- lm(
  y ~ age + sex + bmi + bp + s1 + s2 + s3 + s4 + s5 + s6,
  data = training
)
p_full <- length(coef(model_full))
rss_full <- sum(residuals(model_full)^2)
residual_df_full <- df.residual(model_full)
sigma2_full <- rss_full / residual_df_full

part_a_results <- data.frame(
  model = c("Model A", "Model B", "Full reference"),
  p = c(p_A, p_B, p_full),
  RSS = c(rss_A, rss_B, rss_full)
)

print(part_a_results, digits = 7)
```

```
##            model  p     RSS
## 1        Model A  8 1098909
## 2        Model B  9 1089717
## 3 Full reference 11 1089452
```

```r
print(c(
  full_residual_df = residual_df_full,
  sigma2_full = sigma2_full
))
```

```
## full_residual_df      sigma2_full
##          359.000         3034.684
```

b.

```r
cp_A <- rss_A / sigma2_full - n + 2 * p_A
cp_B <- rss_B / sigma2_full - n + 2 * p_B

aic_A <- n * log(rss_A / n) + 2 * p_A
aic_B <- n * log(rss_B / n) + 2 * p_B

bic_A <- n * log(rss_A / n) + p_A * log(n)
bic_B <- n * log(rss_B / n) + p_B * log(n)

part_b_results <- data.frame(
  model = c("Model A", "Model B"),
  p = c(p_A, p_B),
  RSS = c(rss_A, rss_B),
  Cp = c(cp_A, cp_B),
  reduced_AIC = c(aic_A, aic_B),
  reduced_BIC = c(bic_A, bic_B)
)

print(part_b_results, digits = 7)
```

```
##     model p     RSS       Cp reduced_AIC reduced_BIC
## 1 Model A 8 1098909 8.116253    2974.640    3005.948
## 2 Model B 9 1089717 7.087434    2973.532    3008.754
```

```r
selected_models <- c(
  Cp = part_b_results$model[which.min(part_b_results$Cp)],
  AIC = part_b_results$model[which.min(part_b_results$reduced_AIC)],
  BIC = part_b_results$model[which.min(part_b_results$reduced_BIC)]
)
selected_models
```

```
##        Cp       AIC       BIC
## "Model B" "Model B" "Model A"
```

Mallows' C_p and reduced AIC select Model B, whereas reduced BIC selects Model A.

c.

```r
rss_reduction <- rss_A - rss_B
aic_bic_fit_improvement <- n * log(rss_A / rss_B)
cp_fit_improvement <- rss_reduction / sigma2_full

penalty_comparison <- c(
  RSS_reduction = rss_reduction,
```

```
  AIC_BIC_fit_improvement = aic_bic_fit_improvement,
  Cp_fit_improvement = cp_fit_improvement,
  AIC_extra_penalty = 2,
  BIC_extra_penalty = log(n),
  Cp_extra_penalty = 2
)
print(penalty_comparison, digits = 7)
```

```
##            RSS_reduction AIC_BIC_fit_improvement      Cp_fit_improvement
##              9191.508850                3.107775                3.028819
##       AIC_extra_penalty       BIC_extra_penalty        Cp_extra_penalty
##                2.000000                5.913503                2.000000
```

Adding `s6` reduces RSS, so Model B fits the training data better. On the C_p scale, the improvement in fit is about 3.029, which is greater than the one-parameter penalty of 2; therefore, C_p favors Model B. For reduced AIC and BIC, the improvement in their common fit term is about 3.108. This exceeds AIC's additional penalty of 2, so AIC favors Model B, but it is smaller than BIC's additional penalty of $\log(370) \approx 5.914$, so BIC favors Model A.

Selection by one of these criteria does not prove that the selected model is the true data-generating model. The selection is based on one random sample and only compares the models in the candidate set, while the true relationship may contain interactions that neither candidate model represents.

Question 5

a. Step 2 is the first misuse. The predictor count is selected by the final test data even though the coefficients were fitted by the training data, so the minimum of eleven test MSE values is generally too favorable.

b. We can divide rows 1–370 into five folds: for each candidate predictor count j = 1, 2, . . . , 10, fit the model on four folds and calculate its validation MSE on the held-out fold. Repeat till every fold serves once as validation data, and select the smallest j attaining the lowest mean cross-validation MSE. After that, refit that selected model using all 370 training observations. Rows 371–442 are not examined during model selection and are used only once to calculate the final test MSE of the refitted model.

c. The exploratory analysis can be reported, however, data-dependent preprocessing should be estimated with the four training folds. The final conclusion needs a new, independent dataset.